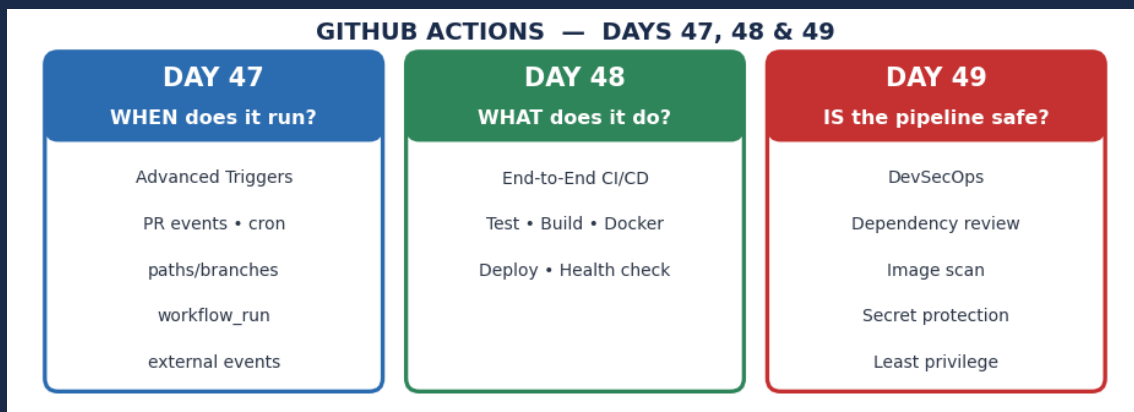


LEARNING NOTES

GitHub Actions

Days 47, 48 & 49

Advanced Triggers → End-to-End CI/CD → DevSecOps



A fully-illustrated edition of the learning notes —
every word preserved, every concept visualized.

DAY
47**Advanced Triggers**

A trigger tells GitHub WHEN to start a workflow.

Simple meaning: Advanced triggers let you react to PR lifecycle changes, schedules, file/branch changes, another workflow finishing, or an outside system sending an event.



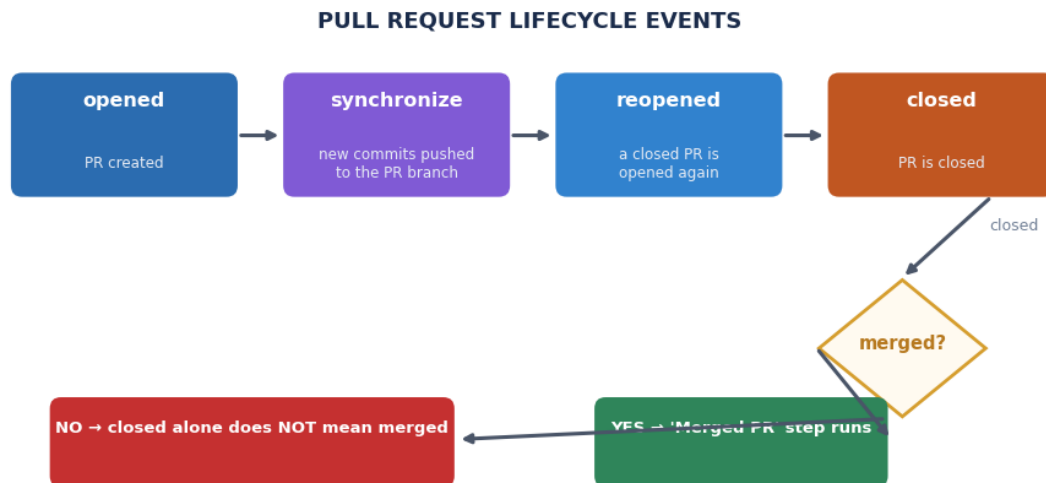
1. Pull Request lifecycle events

opened = PR created. **synchronize** = new commits pushed to the PR branch. **reopened** = a closed PR is opened again. **closed** = PR is closed; it may or may not be merged.

```
name: PR Lifecycle
on:
  pull_request:
    types: [opened, synchronize, reopened, closed]
permissions:
  contents: read
jobs:
  inspect:
    runs-on: ubuntu-latest
    steps:
      - run: |
          echo "Event: ${ github.event.action }";
          echo "Title: ${ github.event.pull_request.title }";
          echo "Author: ${ github.event.pull_request.user.login }";
          echo "Source: ${ github.event.pull_request.head.ref }";
          echo "Target: ${ github.event.pull_request.base.ref }";
      - name: Merged PR
        if: github.event.action == 'closed' && github.event.pull_request.merged == true
        run: echo "PR was merged!"
```

IMPORTANT

The merge condition is important: **closed** alone does not mean **merged**. Check `github.event.pull_request.merged == true` to run steps only for a real merge.



2. PR validation workflow

```

name: PR Checks
on:
  pull_request:
    branches: [main]
permissions:
  contents: read
jobs:
  file-size-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Check files
        run: |
          bad=0
          while IFS= read -r -d $'\n'; file; do
            size=$(stat -c%s "$file")
            if [ "$size" -gt 1048576 ]; then
              echo "Too large: $file"
              bad=1
            fi
          done && (find . -type f -not -path $'\n'/.git/*&\n; -print0)
          exit $bad

  branch-name-check:
    runs-on: ubuntu-latest
    steps:
      - run: |
          b="$github.head_ref"
          if [[ "$b" =~ ^(feature|fix|docs)/.+ ]]; then
            echo "Valid: $b"
          else
            echo "Invalid: $b"; exit 1
          fi

  pr-body-check:
    runs-on: ubuntu-latest
    steps:
      - name: Check PR body
        env:
          BODY: "${github.event.pull_request.body}"
        run: |
          if [ -z "$BODY" ]; then
            echo "::warning::PR description is empty"
          else
            echo "PR description exists"
          fi

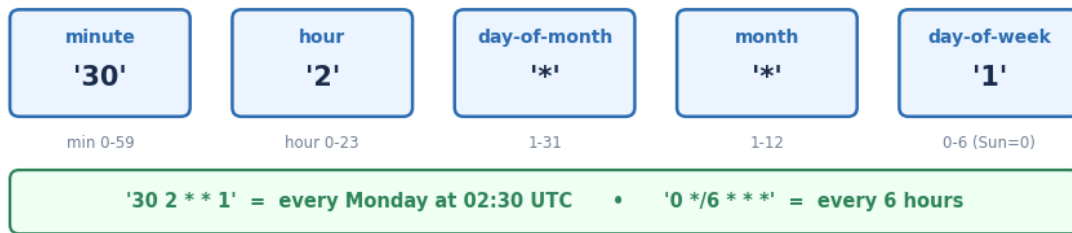
```

TIP

Practice: Make a branch named bad-branch and open a PR — the branch check should fail. Try feature/my-change and check.

3. Scheduled workflows — cron

GitHub Actions cron uses **UTC**. The five fields are: **minute hour day-of-month month day-of-week**.

CRON — 5 FIELDS (GitHub Actions uses UTC)

```
on:
  schedule:
    - cron: '30 2 * * 1'
    - cron: '0 */6 * * *'
  workflow_dispatch:
jobs:
  health:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Triggered by: ${github.event.schedule}"
      - name: URL health check
        run: |
          code=$(curl -L -s -o /dev/null -w "%{http_code}" https://example.com)
          echo "HTTP: $code"
          test "$code" -ge 200 -a "$code" -lt 400
```

Every weekday at 9AM IST:	'30 3 * * 1-5' — because 09:00 IST is 03:30 UTC.
First day of every month, midnight UTC:	'0 0 1 * *'
Midnight IST:	'30 18 * * *' — because IST is UTC+5:30.
Every weekday at 9AM IST:	'30 3 * * 1-5' — because 09:00 IST is 03:30 UTC.
First day of every month, midnight UTC:	'0 0 1 * *'
Midnight IST:	'30 18 * * *' — because IST is UTC+5:30.

WARNING

Scheduled jobs can be delayed during periods of high GitHub Actions load, and GitHub documents conditions under which scheduled workflows may be skipped in inactive repositories. Also remember: scheduled workflows run from the **default branch**.

TIP

`workflow_dispatch` lets you run the scheduled workflow manually from the Actions tab — great for testing without waiting for the timer.

4. Path and branch filters

```

on:
  push:
    branches: [main, &#x27;release/*&#x27;]
    paths:
      - &#x27;src/**&#x27;
      - &#x27;app/**&#x27;

# Example using paths-ignore:
on:
  push:
    branches: [main, &#x27;release/*&#x27;]
    paths-ignore:
      - &#x27;*.md&#x27;
      - &#x27;docs/**&#x27;

```

paths = run only for matching files (narrow allow-list). **paths-ignore** = run unless the changes are only in the ignored files (broad workflow with a few exceptions).

5. workflow_run — connect workflows

```

# Workflow A
name: Run Tests
on:
  push:
    branches: [main]
permissions:
  contents: read
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo &quot;Run real tests here&quot;;

# Workflow B
name: Deploy After Tests
on:
  workflow_run:
    workflows: [&quot;Run Tests&quot;]
    types: [completed]
permissions:
  contents: read
jobs:
  deploy:
    if: github.event.workflow_run.conclusion == &#x27;success&#x27;
    runs-on: ubuntu-latest
    steps:
      - run: echo &quot;Tests passed – deploy here&quot;;

```

workflow_run — CONNECT TWO WORKFLOWS



workflow_run = event AFTER completion | **workflow_call** = REUSABLE workflow called directly

6. repository_dispatch — external event

```
name: External Trigger
on:
  repository_dispatch:
    types: [deploy-request]
permissions:
  contents: read
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Environment: ${github.event.client_payload.environment}"
```

```
gh api repos/<owner>/<repo>/dispatches \
-f event_type=deploy-request \
-f client_payload="{\"environment\":\"production\"}"
# send this from an external system (or your terminal)
```

Use an external trigger when another system — such as an internal deployment tool, monitoring service, or bot — needs to start an approved automation flow. **Protect deployment workflows** that respond to these events.

WHICH TRIGGER DO I USE?

push / pull_request	→ code changes or PR activity
schedule (cron)	→ run on a timer, in UTC
paths / branches	→ only when certain files change
workflow_run	→ after another workflow completes
workflow_call	→ reusable workflow, called directly
workflow_dispatch	→ run manually from the Actions tab
repository_dispatch	→ an outside system triggers it

DAY 47 — PRACTICE CHECKLIST

- ☐ Test **opened**, **synchronize**, **reopened** and **closed** PR events.
- ☐ Verify the merged-only step runs only for a real merge.
- ☐ Test the bad branch name and a valid branch name.
- ☐ Run the scheduled workflow manually with `workflow_dispatch`.
- ☐ Change only a Markdown file and observe a path-filtered workflow.
- ☐ Chain `tests.yml` → `deploy-after-tests.yml` with `workflow_run`.
- ☐ Send `repository_dispatch` with `environment=production`.
- ☐ Save screenshots and YAML in your notes.

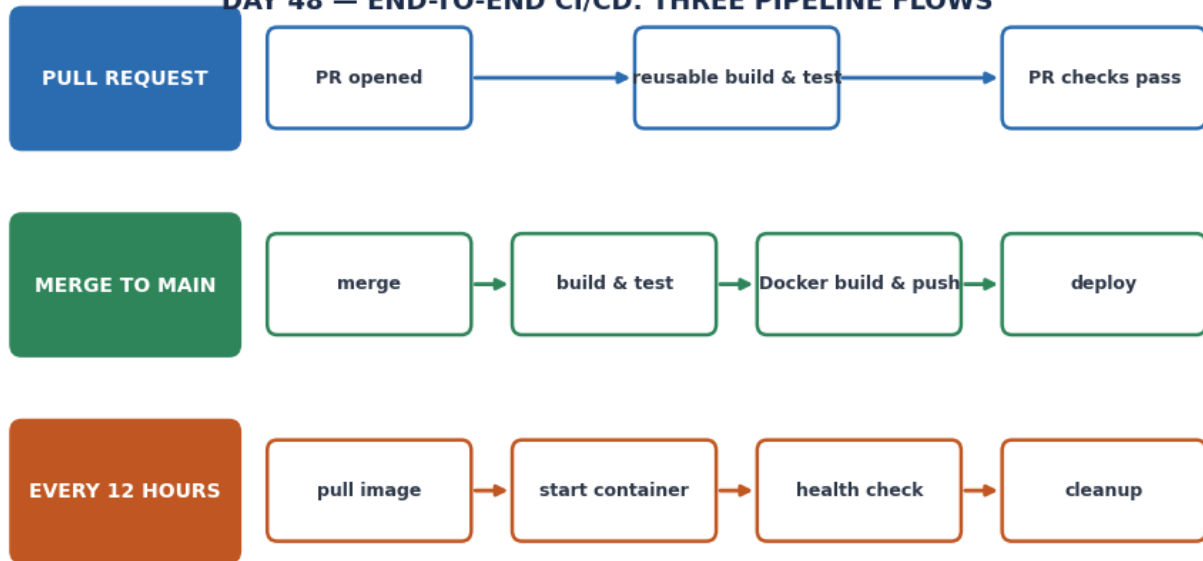
**DAY
48****End-to-End GitHub Actions CI/CD Project**

Combine the skills from earlier days into one capstone.

**HOW TO
STUDY**

Read one section, copy the example into your own repository, run it, change something, and observe the result. The goal is understanding — not memorizing.

The clean mental model: PRs validate → main creates a release → deployment happens after successful stages → scheduled checks monitor.

DAY 48 — END-TO-END CI/CD: THREE PIPELINE FLOWS

PRs validate → main creates a release → deployment after successful stages → scheduled checks monitor

1. Suggested project structure

```

github-actions-capstone/
├── .github/workflows/
│   ├── reusable-build-test.yml
│   ├── reusable-docker.yml
│   ├── pr-pipeline.yml
│   ├── main-pipeline.yml
│   └── health-check.yml
├── Dockerfile
├── app/ or src/
└── tests/
  
```

Use a tiny Flask/FastAPI app, Node/Express app, or your Day 36 Dockerized app. Add a `/health` endpoint so automation can test whether the container is alive.

2. Reusable build & test workflow

```
name: Reusable Build Test
on:
  workflow_call:
    inputs:
      node_version:
        type: string
        required: false
        default: '22';
      run_tests:
        type: boolean
        required: false
        default: true
    outputs:
      test_result:
        description: Test result
        value: ${ jobs.build-test.outputs.test_result }
jobs:
  build-test:
    runs-on: ubuntu-latest
    outputs:
      test_result: ${ steps.result.outputs.status }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${ inputs.node_version }
          cache: npm
      - run: npm ci
      - run: npm run build
      - if: inputs.run_tests
        run: npm test
      - id: result
        run: echo "status=passed" && " $GITHUB_OUTPUT"
```

TIP

For Python, replace the Node setup/install commands with Python equivalents. The reusable-workflow idea stays exactly the same.

3. Reusable Docker build & push workflow

```
name: Reusable Docker
on:
  workflow_call:
    inputs:
      image_name:
        type: string
        required: true
      tag:
        type: string
        required: true
    secrets:
      docker_username:
        required: true
      docker_token:
        required: true
    outputs:
      image_url:
        description: Image URL
        value: ${ jobs.docker.outputs.image_url }
jobs:
  docker:
    runs-on: ubuntu-latest
    outputs:
      image_url: ${ steps.out.outputs.image }
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          username: ${ secrets.docker_username }
          password: ${ secrets.docker_token }
      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${ inputs.image_name }:${ inputs.tag }
      - id: out
        run: echo "image=${ inputs.image_name }:${ inputs.tag }" >> &quot;${GITHUB_OUTPU
```

4. PR pipeline

```
name: PR Pipeline
on:
  pull_request:
    branches: [main]
    types: [opened, synchronize]
permissions:
  contents: read
jobs:
  build-test:
    uses: ../github/workflows/reusable-build-test.yml
    with:
      run_tests: true
  pr-comment:
    needs: build-test
    runs-on: ubuntu-latest
    steps:
      - run: echo "PR checks passed for branch: ${github.head_ref}"
```

NOTE

No Docker push happens here. This keeps PR validation separate from publishing a release artifact.

5. Main branch pipeline

```
name: Main Pipeline
on:
  push:
    branches: [main]
permissions:
  contents: read
jobs:
  build-test:
    uses: ../github/workflows/reusable-build-test.yml
    with:
      run_tests: true
  docker:
    needs: build-test
    uses: ../github/workflows/reusable-docker.yml
    with:
      image_name: YOUR_DOCKER_USERNAME/myapp
      tag: latest
    secrets:
      docker_username: ${secrets.DOCKER_USERNAME}
      docker_token: ${secrets.DOCKER_TOKEN}
  deploy:
    needs: docker
    runs-on: ubuntu-latest
    environment: production
    steps:
      - run: echo "Deploying image after Docker push"
```

TIP

For real releases, also publish an immutable commit tag such as `sha-a1b2c3d`. That gives you **traceability** and makes **rollback** easier than relying only on `latest`.

6. Scheduled health check

```

name: Health Check
on:
  schedule:
    - cron: &#x27;0 */12 * * *&#x27;;
  workflow_dispatch:
permissions:
  contents: read
jobs:
  health:
    runs-on: ubuntu-latest
    steps:
      - run: docker pull YOUR_DOCKER_USERNAME/myapp:latest
      - run: docker run -d --name myapp -p 3000:3000 YOUR_DOCKER_USERNAME/myapp:latest
      - run: sleep 5
      - name: Check health endpoint
        run: |
          code=$(curl -s -o /dev/null -w &quot;{%http_code}&quot; http://localhost:3000/health)
          echo &quot;HTTP: $code&quot;;
          test &quot;$code&quot;; -ge 200 -a &quot;$code&quot;; -lt 400
      - name: Summary
        if: always()
        run: |
          echo &quot;## Health Check Report&quot;; &gt;&gt; &quot;${GITHUB_STEP_SUMMARY}&quot;;
          echo &quot;- Image: myapp:latest&quot;; &gt;&gt; &quot;${GITHUB_STEP_SUMMARY}&quot;;
          echo &quot;- Time: $(date)&quot;; &gt;&gt; &quot;${GITHUB_STEP_SUMMARY}&quot;;
      - name: Cleanup
        if: always()
        run: docker rm -f myapp || true

```

7. Production environment

Create **production** under repository **Settings** → **Environments**. **Required reviewers** can make production deployment wait for approval. This is a useful real-world separation between a successful pipeline and an **authorized production release**.

8. README badges

```

! [PR Pipeline](https://github.com/OWNER/REPO/actions/workflows/pr-pipeline.yml/badge.svg)
! [Main Pipeline](https://github.com/OWNER/REPO/actions/workflows/main-pipeline.yml/badge.svg)
! [Health Check](https://github.com/OWNER/REPO/actions/workflows/health-check.yml/badge.svg)

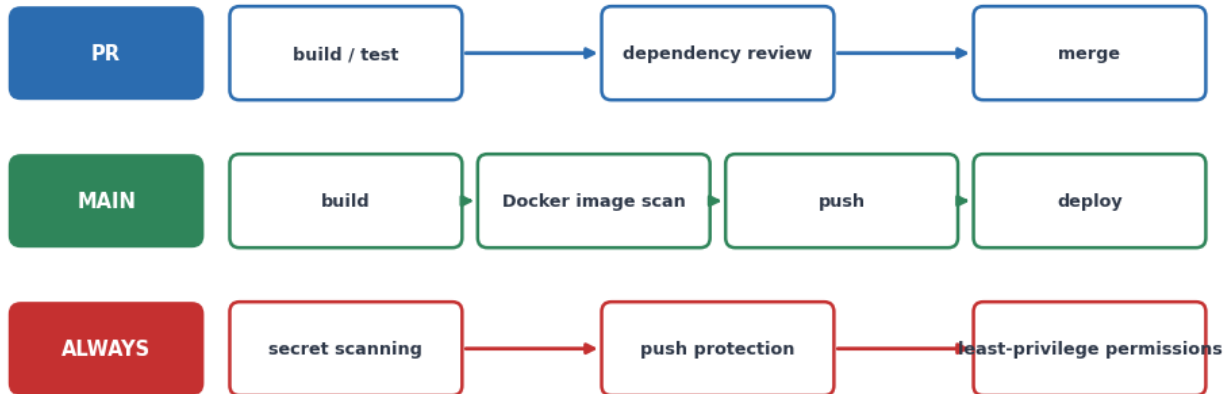
```

DAY 48 — PRACTICE CHECKLIST

- ☐ Build the application and Docker image locally.
- ☐ Open a PR and confirm build/test runs but Docker is **not** pushed.
- ☐ Merge to main and confirm build/test → Docker → deploy.
- ☐ Run health-check manually before waiting for the schedule.
- ☐ Add production approval and test the pause/approval flow.
- ☐ Write what you would add next: notifications, staging, rollback, or multiple environments.
- ☐ Save screenshots and YAML to your notes.

**DAY
49****DevSecOps**

Security checks are built INTO the delivery pipeline — not left until after deployment.

DAY 49 — DEVSECOPS: SECURITY BUILT INTO EVERY STAGE**1. Docker image scanning with Trivy**

```

- name: Scan Docker Image
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: &#x27;YOUR_DOCKER_USERNAME/myapp:latest&#x27;
    format: &#x27;table&#x27;
    exit-code: &#x27;1&#x27;
    severity: &#x27;CRITICAL,HIGH&#x27;
  
```

Meaning: Trivy checks the image for known vulnerabilities. `format: table` makes logs readable.`exit-code: 1` can fail the job when configured findings are detected. Review findings and fix/update the affected packages or base image.**TIP****Practice:** record the base image, vulnerabilities found, what you changed, and the result after re-scanning.**2. Secret scanning and push protection****Secret scanning** looks for credentials that may have been committed. **Push protection** can stop supported secrets before they are pushed. If a real secret leaks, **revoke/rotate it immediately** — deleting the text from a file does not make the credential safe.**IMPORTANT****Never put real passwords, API keys, cloud credentials or tokens into code, screenshots, notes, or chat.****3. Dependency review**

```

- name: Dependency Review
  uses: actions/dependency-review-action@v4
  with:
    fail-on-severity: critical
  
```

Difference from Trivy: dependency review checks dependency **changes introduced by a pull request** (focus: package changes), while an image scan examines the **built container** and its installed software.**4. Least-privilege permissions**

```
permissions:
  contents: read
```

If a workflow must comment on PRs, it may need `pull-requests: write`. Give only the permissions required. If an action is compromised, unnecessary write access can increase the damage it can do.

5. Pin actions for stronger supply-chain control

```
# Convenient: tags can move
uses: actions/checkout@v4

# Stronger control: pin an exact reviewed commit
uses: actions/checkout@<FULL_COMMIT_SHA> # version comment
```

Tags are convenient but can move. A commit SHA identifies one exact revision. Pinning third-party actions is a stronger security practice for higher-assurance workflows.

6. Upload Trivy results as SARIF

```
- uses: aquasecurity/trivy-action@master
  with:
    image-ref: &#x27;YOUR_DOCKER_USERNAME/myapp:latest&#x27;
    format: &#x27;sarif&#x27;
    output: &#x27;trivy-results.sarif&#x27;
- uses: github/codeql-action/upload-sarif@v3
  with:
    sarif_file: &#x27;trivy-results.sarif&#x27;
```

SARIF is a standard security-report format. Uploading it lets GitHub's security tooling display scan findings in a structured way (the Security tab).

7. OIDC — next learning step

OIDC can allow GitHub Actions to obtain **short-lived cloud credentials** instead of storing long-lived cloud access keys. This is an important next step for AWS/GCP/Azure deployments.

DAY 49 — PRACTICE CHECKLIST

- ☐ Add Trivy and inspect the scan results.
- ☐ Enable secret scanning and push protection where available.
- ☐ Add dependency review to the PR pipeline.
- ☐ Add `contents: read` to at least two workflows.
- ☐ Make a harmless dependency change in a PR and inspect the review.
- ☐ Create **day-49-devsecops.md** with your scan result and secure pipeline notes.

FINAL**Revision — Days 47 to 49**

WHEN? → WHAT FLOW? → IS IT SAFE?

DAY 47 = WHEN?	DAY 48 = WHAT FLOW?	DAY 49 = IS IT SAFE?
Choose the right trigger for the event: PR events, cron, paths, workflow_run, external.	Test → build → package → deploy → monitor. PRs validate; main releases; schedules watch.	Scan dependencies → scan image → protect secrets → least privilege.

Quick self-test

- Q1** What is the difference between **pull_request** and **push**?
- Q2** What does `github.event.action` tell you?
- Q3** Why use `workflow_dispatch` with `schedule`?
- Q4** What is the difference between **paths** and **paths-ignore**?
- Q5** What does `workflow_run` do?
- Q6** What does `workflow_call` do?
- Q7** Why should PRs usually test without pushing production images?
- Q8** Why are immutable image tags useful?
- Q9** Dependency review vs Docker image scan — what is different?
- Q10** What is least privilege?
- Q11** Secret scanning vs push protection?
- Q12** Why is OIDC useful for cloud deployments?

Troubleshooting

No run	check trigger, branch, path and activity type.
Cron issue	check UTC and remember schedules can be delayed.
workflow_run issue	workflow name must match the upstream workflow.
Docker issue	check image name, registry login and permissions.
Deployment too early	check needs: or workflow chaining.
Health check failure	inspect port, endpoint, startup time and container logs.
Security failure	inspect the exact package/CVE and update the vulnerable component.
Leaked credential	rotate/revoke it immediately.

Files to create

```

day-47-advanced-triggers.md
day-48-actions-project.md
day-49-devsecops.md

```

**HOW TO
STUDY**

Final rule: Never blindly copy a workflow. Read the trigger, permissions, secrets, job dependencies and commands. **Change one thing, run it, and understand the result.**